

Atividade 2 - Gerência de Configuração de Software e Integração Contínua

Nesta atividade, a equipe deverá configurar e utilizar o repositório do projeto de forma a estabelecer um fluxo controlado, rastreável e automatizado para o desenvolvimento e integração de mudanças. O objetivo é aplicar, na prática, conceitos de **Gerência de Configuração de Software**, **Git**, **gestão de mudanças**, **revisão de código**, **testes automatizados** e **Integração Contínua (CI)**. A atividade deverá ser realizada no mesmo repositório utilizado pelo grupo ao longo da disciplina.

1. Configuração do repositório

A equipe deverá adicionar o professor responsável (cafeio-unicamp) e o PED (Andre-Satorres) da disciplina como colaboradores do repositório, de modo que seja possível consultar o código, Pull Requests e execuções do pipeline. O acesso deverá permitir, quando necessário, a criação de branches e Pull Requests de teste para verificação das regras de proteção. A equipe deverá configurar regras de proteção, por meio de **GitHub Rulesets ou mecanismo equivalente**, para as branches **main** e **develop**, incluindo, minimamente:

- exigir Pull Request para integração de mudanças;
- exigir ao menos uma aprovação realizada por outro integrante da equipe;
- exigir que as discussões de revisão sejam resolvidas antes do merge;
- exigir que os checks obrigatórios do pipeline sejam concluídos com sucesso antes do merge;
- impedir que as regras de proteção sejam ignoradas por usuários com permissão de administração;
- não permitir force push nas branches protegidas;
- não permitir a exclusão das branches protegidas.

2. Desenvolvimento das funcionalidades

Nesta atividade, cada integrante do grupo deverá selecionar e desenvolver **uma funcionalidade pequena e útil para o projeto**.

Para cada funcionalidade:

1. Criar uma issue descrevendo a mudança a ser realizada.
2. Adicionar à issue o label **Atividade2**.
3. Criar, a partir da branch **develop**, uma branch específica para a funcionalidade, utilizando o padrão **feature/nome-da-feature**.
4. Desenvolver a funcionalidade exclusivamente na branch criada.

5. Desenvolver testes automatizados relacionados à funcionalidade.
6. Realizar commits que registrem de forma adequada a evolução da implementação.
7. Abrir um Pull Request da branch **feature/*** para a branch **develop**, relacionando-o à issue correspondente.
8. Solicitar a revisão do Pull Request aos outros integrantes da equipe.
9. Realizar a revisão da contribuição antes da integração.
10. Caso sejam identificados problemas durante a revisão ou pela execução do pipeline, realizar as correções necessárias na própria branch da funcionalidade.
11. Realizar o merge em **develop** somente após a aprovação do Pull Request e a execução bem-sucedida dos checks obrigatórios.
12. Encerrar a issue correspondente, preferencialmente de forma automática por meio da associação com o Pull Request.

Deve ser possível identificar claramente a rastreabilidade entre:

issue → **feature/*** → commits → Pull Request → revisão → **develop**

3. Integração Contínua

A equipe deverá configurar um pipeline de Integração Contínua para o projeto. O pipeline deverá ser executado automaticamente durante o fluxo de desenvolvimento e conter, minimamente, as seguintes verificações.

Build

A aplicação deverá possuir um processo automatizado de construção ou preparação do projeto, de acordo com a tecnologia utilizada. Esse processo deverá:

- instalar ou resolver as dependências necessárias;
- utilizar scripts ou arquivos de configuração versionados no repositório;
- permitir que o projeto seja construído a partir de um clone limpo do repositório.

Exemplos de ferramentas incluem Maven, Gradle, npm, Yarn, pip, Poetry, entre outras.

Análise estática / lint

O pipeline deverá executar alguma ferramenta automatizada de análise estática, lint ou verificação de qualidade do código. Exemplos incluem ESLint, Checkstyle, SpotBugs, Ruff, Pylint ou ferramentas equivalentes. Violações configuradas como erros deverão impedir a conclusão bem-sucedida do pipeline. A ferramenta deverá:

- possuir sua configuração versionada no repositório;
- ser executável de maneira automatizada;
- detectar violações de qualidade previamente definidas pela equipe;

- ocasionar falha da verificação quando forem encontrados erros configurados como impeditivos.

Testes automatizados

Todos os testes do repositório, incluindo os desenvolvidos para as funcionalidades da atividade deverão ser executados automaticamente pelo pipeline. Caso algum teste falhe, o pipeline também deverá falhar.

Cobertura de testes

O pipeline deverá calcular a cobertura dos testes automatizados utilizando uma ferramenta adequada à tecnologia adotada. O resultado da cobertura deverá estar disponível para consulta na execução do pipeline ou por meio de relatório gerado durante a execução.

Ferramentas sugeridas

As ferramentas abaixo são apenas sugestões. A equipe poderá utilizar ferramentas equivalentes adequadas à tecnologia escolhida, desde que atendam aos requisitos da atividade.

Tecnologia	Cobertura de testes	Análise estática / lint
Java / Kotlin	JaCoCo	Checkstyle, SpotBugs
Python	pytest-cov	Ruff, Pylint
JavaScript / TypeScript	Vitest/Jest com coverage, Istanbul/nyc	ESLint
C / C++	gcov/lcov	clang-tidy
.NET / C#	Coverlet	analyzers do .NET

Documentação:

- Java/Kotlin: [JaCoCo](#) · [Checkstyle](#) · [SpotBugs](#)
- Python: [pytest-cov](#) · [Ruff](#)
- JavaScript/TypeScript: [Vitest Coverage](#) · [ESLint](#)
- C/C++: [gcov](#) · [clang-tidy](#)
- .NET/C#: [Coverlet](#)

4. Integração das mudanças

O pipeline deverá ser executado **automaticamente nos Pull Requests antes que as alterações possam ser integradas, bem como após a integração de mudanças nas branches `develop` e `main`**. A configuração do repositório deverá impedir o merge enquanto os checks obrigatórios do Pull Request não forem concluídos com sucesso. Após o merge, uma nova execução automática do pipeline deverá validar o estado resultante da branch de destino.

5. Release e baseline

Após a integração das funcionalidades da atividade na branch `develop`, a equipe deverá preparar a versão a ser entregue seguindo o fluxo de release adotado no projeto. Para isso, a equipe deverá:

1. criar uma branch `release/*` a partir da branch `develop`;
2. realizar, nessa branch, apenas os ajustes necessários para preparação da versão, sem inclusão de novas funcionalidades;
3. abrir um Pull Request da branch `release/*` para a branch `main`;
4. realizar o merge somente após a aprovação do Pull Request e a execução bem-sucedida dos checks obrigatórios;
5. criar, **no commit correspondente à versão integrada em `main`**, a tag `a2-v1.0.0`;
6. atualizar a branch `develop` com as alterações presentes na `main` após a conclusão da release.

A tag identificará oficialmente a versão entregue e será considerada a **baseline da Atividade 2**. Para fins de avaliação, será considerado o estado do projeto identificado por essa tag. Alterações realizadas posteriormente não serão consideradas na correção. A ausência da tag `a2-v1.0.0` no repositório implicará **nota zero na atividade**.

6. Participação da equipe

Todos os integrantes deverão participar efetivamente do desenvolvimento da atividade e possuir contribuições identificáveis no repositório. Cada integrante deverá, no mínimo:

- ser responsável pelo desenvolvimento de uma das funcionalidades selecionadas;
- possuir commits relacionados à implementação dessa funcionalidade;
- desenvolver ou contribuir para os testes automatizados correspondentes;
- participar do fluxo de Pull Request associado à funcionalidade;
- realizar a revisão de pelo menos um Pull Request de outro integrante da equipe.

Além disso, poderão ser consideradas como evidências complementares de participação:

- criação ou atualização de issues;
- correções realizadas a partir de comentários de revisão;

- configuração das regras de proteção do repositório;
- configuração ou manutenção do pipeline de Integração Contínua;
- preparação da release e da entrega.

Entrega

A entrega deverá ser realizada por apenas um dos integrantes do grupo, por meio do formulário disponibilizado no Classroom. Não é necessário produzir um relatório descrevendo os passos realizados. As evidências devem estar disponíveis diretamente no histórico e nas configurações do repositório.

Critérios de avaliação

Critério	Peso
C1: Configuração e proteção do repositório	20%
C2: Fluxo de desenvolvimento e revisão	30%
C3: Testes automatizados e cobertura	20%
C4: Qualidade estática	10%
C5: Integração contínua	10%
C6: Release e baseline	10%

Cada critério será avaliado em quatro níveis:

- **Ótimo (10):** atende plenamente ao objetivo do critério, de forma clara, específica, consistente e adequada ao contexto.

- **Bom (7,5):** atende adequadamente ao objetivo, mas apresenta pequenas lacunas ou oportunidades de melhoria.
- **Insuficiente (5):** atende parcialmente ao objetivo, com lacunas relevantes, caráter excessivamente genérico ou problemas conceituais.
- **Ausente/Inadequado (0):** não apresenta evidência suficiente ou apresenta problemas que comprometem significativamente o objetivo do critério.

C1: Será avaliado se as branches protegidas possuem as regras exigidas e se essas regras controlam efetivamente a integração de mudanças.

C2: Será avaliada a rastreabilidade e aplicação do fluxo definido no item 2.

C3: Será avaliado se:

- cada funcionalidade possui testes automatizados relacionados ao comportamento implementado;
- os testes são relevantes e verificam comportamentos significativos da funcionalidade;
- os testes fazem parte do repositório e podem ser executados de forma automatizada;
- a cobertura de testes é calculada por ferramenta apropriada;
- o resultado da cobertura pode ser consultado na execução do pipeline ou em relatório gerado.

C4: Será avaliada a utilização de ferramenta automatizada de lint, análise estática ou verificação de qualidade adequada à tecnologia do projeto.

C5: Será avaliado se o pipeline de Integração Contínua é executado automaticamente nos eventos definidos, contempla as verificações exigidas e está associado às regras de proteção das branches.

C6: Será avaliado o processo de release, integração em **main**, atualização de **develop** e criação da tag.

Nota individual

Os critérios acima determinam inicialmente a nota da equipe. Entretanto, cada integrante deverá apresentar evidências mínimas de participação individual. A nota individual poderá ser ajustada conforme o nível de participação observado no repositório:

- **Participação completa:** 100% da nota da equipe;
- **Participação parcial:** até 80% da nota da equipe;
- **Participação substancialmente incompleta:** até 50% da nota da equipe;
- **Ausência de participação efetiva no desenvolvimento:** nota zero.